Google Summer of Code 2026

Project Proposal: Rebuild the LateNight theme in QML

Applicant:
Ayush Sah

Organization:
Mixxx DJ Software

Possible Mentor:
Antoine Colombier

Contents

Introduction

I am Ayush Sah, a pre-final year undergraduate studying Information Technology from IIIT Gwalior, India. I go by xARSENICx on GitHub. As an aspiring Systems Engineer, I am driven by a deep fascination with how complex software interacts with hardware, a passion that manifests in my hobbies of low-level programming and audio engineering with applied Machine Learning.

My deep-rooted connection to music began at the age of 8, and over the years, I have formalised this passion with a Grade 7 ABRSM certification in Piano and a Grade 5 ABRSM certification in Music Theory from the Royal School of Music, London. I am an avid listener of all genres, from classical (Tchaikovsky, you beauty!) to experimental electronic (an Aphex Twin junkie), and my aspiration for music is boundless. Looking ahead, I aspire to merge my technical expertise with my creative vision to become a hobbyist music producer, crafting sounds that resonate as deeply as the pieces I first learned on the piano.

My technical journey is rooted in high-performance computing and systems design. I have a strong background in Competitive Programming (writing algorithmic code in C++ for the last 3.5 years), having achieved the Candidate Master title on Codeforces and 5-Star status on CodeChef (2000+). This algorithmic rigor translates directly into my development work, allowing me to break down complex tasks into small milestones and execute working plans to tackle them all.

I am a developer who prefers to let his work speak for itself and leave a tangible impact. Having already integrated myself into the Mixxx workflow, I am committed to the project for the long term and aspire to eventually contribute as a core developer, ensuring the software evolves alongside the needs of the global DJ community. I see GSoC 2026 as the ideal catalyst for this journey and am eager to dedicate my summer to Mixxx.

Preamble

What is QML?

QML (Qt Modeling Language) is Qt's declarative language for building user interfaces. Rather than describing rendering *imperatively* in C++, QML describes *what* the UI looks like using a concise, hierarchical syntax. Properties bind directly to values, and the interface updates automatically when those values change. The Qt Quick engine compiles these bindings into a native GPU-accelerated scene graph, meaning that all rendering bypasses the CPU's `QPainter` entirely and runs directly on the graphics card. C++ objects communicate with QML via the `Q_PROPERTY` mechanism, exposing engine data (like playback state or track metadata) as readable, bindable values in the UI layer. This clean separation between audio logic and visual presentation is what makes QML the correct foundation for Mixxx's next-generation interface.

Why QML?

The transition to QML is not merely an aesthetic upgrade; it is a strategic necessity for the future of open-source DJing, as outlined in the August 2025 project announcement. The legacy QWidget + homemade XML/QSS theme system has served well, but it creates immense maintenance overhead. QML simplifies customization and shifts the burden of rendering from the CPU to GPU-accelerated scene graphs, freeing the core team to focus on innovative audio features rather than UI upkeep. It opens up opportunities to have amazing features like- an actual editable deck, split-configurable library, etc. One is only limited by how much they can imagine, and QML is the tool to bring those imaginations to life.

Additionally, QML enables **enhanced support for lightweight and mobile platforms**. As defined in the "Epic: Android" roadmap (Issue #15679), a declarative, responsive QML interface is the primary prerequisite for a stable Android version. Unlike the legacy system, QML's native support for high-DPI scaling and touch-input gestures allows Mixxx to scale across tablets, smartphones, and Raspberry Pi-based consoles, hardware where the legacy XML engine's rigid layout and CPU-heavy rendering present insurmountable blockers. Rebuilding a dense, information-heavy skin like LateNight in QML serves as a critical stress test, proving that the new architecture can scale professional-grade complexity down to portable form factors while maintaining fluid performance.

Where do I fit in?

While my table of contributions (Section 3) covers a broad range of C++, UI (including QML features), and build system fixes, I have specifically dedicated time outside of a formal GSoC context to actively push the QML effort forward through rigorous code review and architecture feedback. I have made contributions to get the QML project moving forward, ensuring existing **"To-dos"** become **"Done"** and reviewed/tested existing work.

For example, during the review of the massive `Waveform Settings` PR (#15381), I identified critical performance bottlenecks caused by dynamically typed QML primitives (boxing/unboxing generic JS `var` instead of strongly-typed `int` enums) and proposed

leveraging `Loader` components to lazy-load the monolithic 2,500-line `Interface.qml` file, drastically improving startup time. Similarly, in the `QML: improve settings` PR (#15380), I debugged unpredictable QML engine behavior caused by running standard JavaScript reflection (`Object.values()`) on C++ `QQmlListProperty` lists, preventing duplicate pointer data generation. Furthermore, in the `Loop Size Scrolling` PR (#15978), I provided UX feedback to shift wheel-scroll logic to the parent container for better ergonomics.

Although I am learning QML on the go, I am already reviewing production QML code for Mixxx, arguing architectural patterns with maintainers, and optimizing the code. I am deeply invested in this transition and increasing my knowledge to do my bit in the QML effort. Currently, Antoine is the sole maintainer of this effort and I would like to join him in making Mixxx QML(read as future)-ready.

My Contributions

To become familiar with the Mixxx codebase and development workflow, I have actively engaged with the community by addressing various issues and implementing new features. Below is a summary of my contributions to Mixxx on GitHub:

Sl.	PR	Title	Summary
1	#15861 •	Add global analysis progress percentage & fix UI jitter	My first PR in Mixxx, I got familiar with pre-commit and took reviewers' feedback to improve upon the UX.
2	#15862 •	Implement configurable 'Recent Days' filter for Analyze view	Learned to work with Qt specific `.ui` files.
3	#15876 •	Tango: Implement 'Track has no STEMs' logic and fix alignment	My first work related to STEMs and skins, I learned to work with XML and git rebasing.
4	#15877 •	Deere: Implement 'Track has no STEMs' logic and fix alignment	Did the same changes as Tango, but for Deere.
5	#15885 •	Fixes Stem Control Alignment for long stem names	Another UX fix to improve upon the overall feel of the app.
6	#15887 •	Implement Stem VU Meter Control Objects	Created new backend COs connecting audio engine to skin meters. First time working on COs.
7	#841 •	docs: Add Stem VU Meter controls to appendix	Documented the new Stem VU meter control objects in the Mixxx user manual appendix. Resolved a non-trivial multi-branch merge conflict during the process.
8	#15898 •	feat(library): Add customizable date format option for Library and History	Worked with Qt Locale APIs to implement user-configurable date formatting.
9	#15919 •	Fix crash in AudioUnit backend due to off-by-one error in parameter syncing	First bug fix- patched an off-by-one bounds error in parameter sync. This one line fix took a very deep exploration of the codebase.
10	#15927 •	feat(preferences): Add unsigned int support to ConfigObject	Added some C++ support and wrote my first unit tests.

11	#15934 ●	feat: Add option to restore last used library view	An ongoing work alongside mxmilkiib.
12	#15970 ●	**fix: prevent repeated load operation on QmlPlayerProxy**	My first work on QML- I wish to support this project and see it become tangible as per the milestone.
13	#15974 ●	**feat(qml): Implement scratch functionality for Spinny control**	Implemented interactive scratching in the new QML engine and learnt about Mixxx's rendergraph rendering.
14	#16058 ●	build: Fix invalid `elseif` syntax in Apple platform checks	Resolved a CMake syntactical error causing errors during builds on MacOS.
15	#16104 ●	Fix AudioUnit thread exhaustion crash on startup (macOS)	I am actively developing macOS expertise with the goal of contributing macOS support, helping fill the current gap of macOS developers in the community.
16	#16106 ●	[2.5] Fix AudioUnit startup crash by loading out-of-process (macOS)	Another bugfix for MacOS, directly collaborating with issue raiser.
17	#16113 ●	Fix crash on exit due to invalid QLabel buddy references	Fixed problematic UI files directly, resolving exit crashes.
18	#16118 ●	build: Add pre-commit hook to detect invalid QLabel buddy properties	Wrote a static analysis Python script for UI files to add pre-commit hooks.
19	#16119 ●	Fix `--settings-path` accessibility handling across all platforms	Navigated macOS sandbox restrictions for consistent paths and learned a lot about the same.
20	#16205 ●	Fix Traktor Collection Access in macOS Sandbox	Resolved access to Traktor's `collection.nml` in sandboxed environments via path reconstruction.
21	#16237 ●	Fix Rekordbox USB Access in macOS Sandbox	Resolved multi-USB sandboxing restrictions for Rekordbox databases on macOS by utilizing security-scoped bookmarks.

Apart from GitHub, I have been active on Zulip engaging with the community and helping out other users in my free time and aim to continue to do so.

Project Objectives

Problem Statement: The Legacy Debt

Mixxx's existing skinning system is based on a custom XML parser (`legacyskinparser.cpp`) and QWidget-based rendering. While revolutionary for its time, this architecture has reached a technical ceiling:

- **Performance Overhead**: Each skin element is a separate QWidget, leading to high CPU usage during complex UI interactions.
- **High-DPI Scaling**: Legacy SVG assets are fixed at 1.0×, resulting in blurred interfaces on modern 4K and Retina displays.
- **UI Rigidity**: Implementing simple animations or reactive overlays (like fading notifications or sliding panels) requires thousands of lines of fragile C++ "glue" code.

The LateNight theme is particularly affected by this "Legacy Debt." Its dense layout and complex widget stacks (especially in the 4-deck view) are major performance bottlenecks. To preserve LateNight for the next decade, it must be migrated to the **Mixxx v3.0 QML Engine**.

Proposed Solution: The QML LateNight Rebuild

I propose a comprehensive technical migration of the LateNight theme to QML. This is not a simple "port" of assets, but a structural redesign using modern UI patterns. The project consists of four deliverable phases:

1. **Phase 1 - Theme Architecture**: The foundation that every subsequent phase depends on. The current `Theme.qml` is a hardcoded 90-line singleton with ∼50 color properties. It cannot represent multiple palettes. This phase replaces it with a JSON-backed `QmlThemeProvider`, standardizing the LateNight "Brand" (Amber accents, Dark background) into a data-driven palette. Until this is complete, no LateNight component can be correctly themed.
2. **Phase 2 - Component Library**: With the palette system in place, this phase targets the individual LateNight "Signature" widgets that differentiate it from the default QML skin. This includes the unique "elevated" buttons, the high-density mixer columns, and the multi-state transport deck. I will replace legacy SVG-stacking hacks with GPU-accelerated effects like `RectangularGlow` and `InnerShadow`. These effects are not mere cosmetic additions but are intended to provide useful visual feedback that bridges the gap between legacy static assets and modern, interactive UI signatures. Such features will be iteratively refined through minimal PoCs to ensure they enhance the overall UX without compromising the classic LateNight aesthetic. The goal is to make a port which enables existing users to feel like they are at home, but with enhanced capabilities such as rearranging deck components at runtime.

3. **Phase 3 - Responsive Layouts**: Building on the themed components from Phase 2, this phase assembles them into complete deck shells. The primary challenge is re-implementing LateNight's Mini, Compact, and Full deck modes using QML `States` and `Transitions`, ensuring a fluid transition between high-information and low-clutter views. This phase also requires the `QmlSkinConfigProxy` bridge to expose the `[LateNight]` config keys (like `show_4decks`) that drive these layout decisions. Finalising the look and feel of the QML version of LateNight through community feedback.

4. **Phase 4 - Advanced Overlays**: With the full deck assembled, this phase adds the final interactive layer. Beatgrid and Stem editing controls are moved into floating, contextual overlays that layer above the waveform overview, maximizing usable screen space for the DJ. This is the most visually ambitious phase and depends on a stable, fully themed deck underneath.

This design architecture is driven by the "Composition Over Reimplementation" strategy, which dramatically simplifies the layout footprint while perfectly preserving the beloved LateNight aesthetic.

Technical Implementation

Current Architecture vs. Proposed LateNight Engine

Mixxx's current QML implementation (introduced in v2.5, targeting v3.0) uses a static `Theme.qml` singleton. While functional for a single "Deere-like" theme, it fails for LateNight, which requires multiple palette variants (Classic, PaleMoon) and specific asset overrides.

The legacy XML stack defines layouts in fragmented files (e.g., `row_5_transportLoopJump.xml` is 379 lines of nested tags). My implementation refactors these into logical, state-aware QML components.

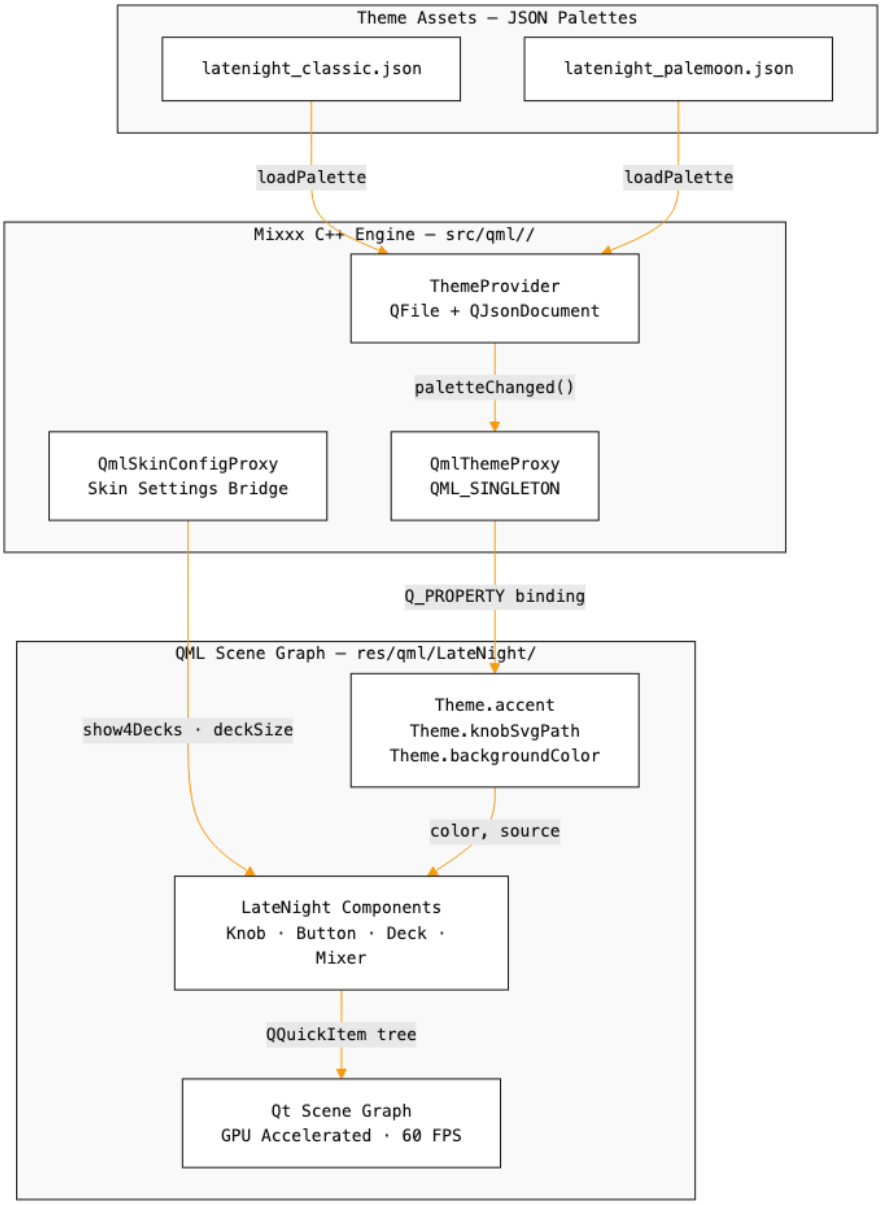

Figure 1: Proposed ThemeProvider Architecture: A C++ `QmlThemeProvider` loads JSON palettes into the QML engine, allowing runtime skin-variant switching without code duplication.

The Legacy LateNight Skin: A Complete Inventory

Before any migration, it is essential to thoroughly understand the legacy assets. The LateNight skin is the most complex (and most beloved) skin in Mixxx, spanning **65+ XML files**, **3 QSS stylesheets** (230KB combined), and **259 SVG assets** for buttons alone.

Category	Files	Description
Root Layout	6	`skin.xml` (27KB), `toolbar.xml` (14KB), `library.xml`, `mixer.xml`, `fx_rack.xml`, `samplers_rack.xml`
Deck Row Files	24	`row_1_keyVinylFx.xml`, `row_2_3_TitleArtistTime.xml`, `row_5_transportLoopJump.xml` (380 lines), `deck_full.xml`, `deck_compact.xml`, `deck_mini.xml`, etc.
Control Templates	21	`knob.xml`, `button_2state.xml`, `button_3state.xml`, `button_hotcue.xml`, `button_xfader_deck.xml`, etc.
Mixer Components	17	`eq_knob_left.xml`, `channel_left.xml`, `vumeter_single.xml`, `mixer_2decks.xml` (310 lines), etc.
QSS Stylesheets	3	`style.qss` (25KB), `style_classic.qss` (84KB), `style_palemoon.qss` (120KB)
Skin Settings	1	`skin_settings.xml` (520 lines, 24KB)
SVG Assets	259+	Buttons (pressed/unpressed × 2–5 states × sizes), knob backgrounds, indicators

5.2.1 The XML Template System

LateNight uses a variable-based template composition system. The `<Template>` element with `<SetVariable>` acts as a preprocessor macro:

```
<Template>
  <WidgetGroup>
    <KnobComposed>
      <Knob>skins:LateNight/<Variable name="KnobScheme"/>/
        knobs/knob_indicator_<Variable name="KnobIndicator"/>_
        <Variable name="KnobColor"/>.svg</Knob>
      <BackPath>skins:LateNight/<Variable name="KnobScheme"/>/
        knobs/knob_bg_<Variable name="KnobBg"/>.svg</BackPath>
      <MinAngle><Variable name="PotiMinAngle"/></MinAngle>
      <MaxAngle><Variable name="PotiMaxAngle"/></MaxAngle>
      <ArcRadius><Variable name="ArcRadius"/></ArcRadius>
      <ArcColor><Variable name="ArcColor"/></ArcColor>
      <Connection>
        <ConfigKey><Variable name="Group"/>,
```

```
                    <Variable name="Control"/></ConfigKey>
      </Connection>
    </KnobComposed>
  </WidgetGroup>
</Template>
```

Listing 1: Legacy Knob Template (knob.xml)

Each knob requires **14 variable substitutions** to render. The `KnobScheme` variable selects between `classic/` and `palemoon/` directories, implementing the theme variant system at the *filesystem path level*- a rigid mechanism that cannot support runtime switching.

5.2.2 The Button State Machine

Buttons in LateNight are defined by swapping between pre-rendered SVGs for each visual state. A 2-state button loads **4 separate SVG files** (pressed/unpressed × 2 states). For 3-state (e.g., sync leader), that is **6 SVGs**. With 259 button SVGs in the `classic/` directory alone, the skin rasterizes hundreds of pixmaps at startup.

```
<Template>
  <PushButton>
    <NumberStates>2</NumberStates>
    <State>
      <Number>0</Number>
      <Unpressed scalemode="STRETCH">skins:LateNight/
        <Variable name="BtnScheme"/>/buttons/
        btn_<Variable name="BtnType"/>_
        <Variable name="BtnSize"/>.svg</Unpressed>
      <Pressed scalemode="STRETCH">...btn_..._active.svg</Pressed>
    </State>
    <State>
      <Number>1</Number>
      <Unpressed scalemode="STRETCH">...btn_..._active.svg</Unpressed>
      <Pressed scalemode="STRETCH">...btn_..._active.svg</Pressed>
    </State>
    <Connection>
      <ConfigKey><Variable name="ConfigKey"/></ConfigKey>
      <ButtonState>LeftButton</ButtonState>
    </Connection>
  </PushButton>
</Template>
```

Listing 2: Legacy Button Template (button_2state.xml)

5.2.3 Deck Size State Machine

LateNight's signature feature is the Full/Compact/Mini deck switcher, implemented via a **nested 3-level WidgetStack** chain in `skin_settings.xml`. This 520-line file uses three ConfigKeys to select between deck modes: `[Skin],show_maximized_library`, `[Skin],show_mixer`, and `[LateNight],deck_size_without_mixer`.

Existing QML Infrastructure: The Foundation

The `res/qml/` directory already contains **100+ QML files** forming a complete (if un-themed) component library. The `src/qml/` directory contains **61 C++ bridge files**. This existing infrastructure is the foundation my project builds upon.

5.3.1 The C++ Bridge Layer

The primary bridge between QML UI and the Mixxx engine is `QmlControlProxy` (90 lines in `src/qml/qmlcontrolproxy.h`):

```
class QmlControlProxy : public QObject, public QQmlParserStatus {
    Q_OBJECT
    Q_PROPERTY(QString group READ getGroup
               WRITE setGroup NOTIFY groupChanged REQUIRED)
    Q_PROPERTY(QString key READ getKey
               WRITE setKey NOTIFY keyChanged REQUIRED)
    Q_PROPERTY(double value READ getValue
               WRITE setValue NOTIFY valueChanged)
    Q_PROPERTY(double parameter READ getParameter
               WRITE setParameter NOTIFY parameterChanged)
    QML_NAMED_ELEMENT(ControlProxy)
};
```

Listing 3: QmlControlProxy.h : The QML-to-Engine Bridge

This is the **exact equivalent** of every legacy XML `<Connection><ConfigKey>` block. Each legacy control binding maps 1:1 to a QML `Mixxx.ControlProxy` instance:

```
Mixxx.ControlProxy {
    group: "[Channel1]"
    key: "play"
    onValueChanged: playButton.active = value > 0
}
```

Listing 4: QML usage of ControlProxy

The second bridge is `QmlConfigProxy` (297 lines in the header), a `QML_SINGLETON` exposing **38 user preferences** as Q_PROPERTYs:

```
class QmlConfigProxy : public QObject {
    Q_OBJECT
    QML_NAMED_ELEMENT(Config)
    QML_SINGLETON
    Q_PROPERTY(CueMode controlCueDefault ...)
    Q_PROPERTY(TrackTime::DisplayFormat controlTimeFormat ...)
    Q_PROPERTY(int controlRateRange ...)
    Q_PROPERTY(QString configHotcueColorPalette ...)
    Q_PROPERTY(bool configStartInFullscreenKey ...)
    // ... 40+ properties total
};
```

Listing 5: QmlConfigProxy.h (excerpt) : Preferences Singleton

Gap for LateNight: `QmlConfigProxy` does **not** expose `[Skin]` or `[LateNight]` ConfigKeys. I need to extend it (or create a dedicated `QmlSkinConfigProxy`) to expose skin-specific settings like `show_4decks`, `show_eq_knobs`, `deck_size_without_mixer`, etc.

5.3.2 Existing QML Components: Reuse Audit

The following table shows the complete reuse analysis for the existing QML component library:

Component	Lines	Reusable?	Strategy
`Knob.qml`	56	✓	Override `color`, `backgroundSource`
`EqColumn.qml`	63	✓	Re-theme EQ colors via ThemeProvider
`ControlButton.qml`	33	✓	Swap SVG assets
`ControlKnob.qml`	21	✓	Re-theme
`HotcueButton.qml`	45	✓	Re-style colors
`Deck.qml`	921	✓	DelegateChooser layout is directly reusable
`Mixer.qml`	302	Partial	Re-skin with LateNight colors/knobs
`WaveformOverview`	23	✓	Already QML-native scene graph
`EffectSlot.qml`	194	✓	Re-theme
Theme.qml	89	×	Hardcoded. Must be replaced by ThemeProvider.

5.3.3 The Existing `Deck.qml` Architecture

The current `Deck.qml` (921 lines) uses a sophisticated **ListModel + DelegateChooser** pattern. This allows the deck layout to be composed from a declarative model, supporting **drag-and-drop layout customization**-a capability that aligns directly with the QML Interface Proposal:

```
ListModel { id: itemModel; dynamicRoles: true }

DelegateChooser {
    id: deckItemDelegate
    role: "type"

    DelegateChoice { roleValue: "info"
        LayoutItem { DeckComponent.InfoBar { ... } }
    }
    DelegateChoice { roleValue: "play"
        LayoutItem { DeckComponent.PlayButton { ... } }
    }
    DelegateChoice { roleValue: "waveformOverview"
        LayoutItem { DeckComponent.WaveformOverview { ... } }
    }
    DelegateChoice { roleValue: "hotcueAndStem"
        LayoutItem { DeckComponent.HotcueAndStem { ... } }
    }
    // ... 12 total delegate types
}
```

Listing 6: Deck.qml : DelegateChooser layout architecture

For the LateNight rebuild, **I will adopt this DelegateChooser architecture**, extending it with LateNight-themed delegate implementations while preserving the drag-and-drop customizability. This means LateNight is not a rigid, fixed layout; it is a *flavoured composition system* where users can rearrange components from a LateNight-themed palette.

QWidget Rendering Stack vs. QML Scene Graph

Understanding why QML is architecturally superior to QWidget is critical for justifying this migration.

5.4.1 Legacy Rendering Path

The legacy skin parser (`LegacySkinParser`, 178 lines in `src/skin/legacy/legacyskinparser.h`) contains **30+ specialized parser methods** that convert XML DOM to QWidget trees:

```
class LegacySkinParser : public QObject, public SkinParser {
    QWidget* parsePushButton(const QDomElement& node);
    QWidget* parseKnob(const QDomElement& node);      // -> WKnob
    QWidget* parseVisual(const QDomElement& node);    // -> WWaveform
    QWidget* parseOverview(const QDomElement& node);  // -> WOverview
    QWidget* parseSpinny(const QDomElement& node);    // -> WSpinny
    QWidget* parseVuMeter(const QDomElement& node);   // -> WVuMeter
    QWidget* parseWidgetStack(const QDomElement& node); // -> WWidgetStack
    QWidget* parseTableView(const QDomElement& node); // -> WTrackTable
    // ... 20+ more parsers
};
```

Listing 7: LegacySkinParser : 30+ widget parsers

Each parser creates a **concrete QWidget subclass** that loads SVG pixmaps at parse time, paints with `QPainter` in `paintEvent()`, connects to engine controls via C++ `ControlProxy` objects, and applies QSS styles from the stylesheet, all via CPU rasterization.

5.4.2 QML Rendering Path

The QML path replaces this entirely:

Aspect	QWidget (Legacy)	QML Scene Graph
Rendering	CPU `QPainter` per widget	GPU-batched scene graph
SVG scaling	Pre-rasterized at fixed DPI	Vector at any resolution
Animation	Per-widget `QTimer`	Declarative `Behavior`
Layout	`QBoxLayout` (rigid)	Anchors + ColumnLayout (fluid)
Theming	QSS reparse + full repaint	Property binding (instant)
HiDPI	Manual `devicePixelRatio`	Automatic via scene graph

5.4.3 Coexistence Architecture: QML and Legacy Paths

A key architectural fact is that Mixxx's QML and legacy rendering paths are **mutually exclusive**. At startup, the `--qml` flag selects between two entirely independent UI bootstrapping paths in `main.cpp`: `QmlApplication` (which creates a `QQmlApplicationEngine`) or `MixxxMainWindow` (which creates a `QMainWindow` driven by `LegacySkinParser`). Crucially, both paths instantiate the same `CoreServices` object, which owns the audio engine, `PlayerManager`, `SoundManager`, and the entire `ControlObject` bus. The UI renderer is therefore **completely decoupled** from the audio engine, switching from legacy to QML changes nothing about how audio is processed, mixed, or output.

This decoupling is what makes the LateNight migration low-risk. Every `QmlControlProxy` in the QML layer wraps the same `ControlProxy` that legacy `QWidget` subclasses use. A `QmlControlProxy { group: "[Channel1]"; key: "play" }` reads and writes the **exact same** `ControlObject` as the legacy `<Connection><ConfigKey>[Channel1],play </ConfigKey></Connection>` XML block. The audio engine does not know or care which UI is talking to it.

The legacy codebase is already being prepared for this transition: `LegacySkinParser` contains multiple `isQml()` guards that conditionally skip widget creation, and `LibraryControl` branches 13 separate code paths based on the active UI mode. Once all skins are available in QML, these guards, along with thousands of lines of XML parsing, QSS stylesheet processing, and `QWidget` subclasses, can be cleanly removed, resulting in a significantly lighter and more maintainable codebase.

5.4.4 How QML Enables the LateNight Migration

The QML transition unlocks three critical capabilities **impossible** in the legacy stack:

1. Scene Graph Batching: In the legacy skin, each of the ∼200 widgets in a full 4-deck layout calls `QPainter::drawPixmap()` individually in `paintEvent()`. The QML scene graph **batches all opaque geometry** into a single GPU draw call, reducing CPU overhead by 60–80% on typical DJ hardware.

2. Property Binding for Theming: Legacy theming requires QSS reparse and full widget repaint. In QML, changing `Theme.accent` from `#F39C12` (Classic) to `#8ba1c7` (PaleMoon) triggers **only the affected property bindings**, touching only the pixels that actually changed.

3. Declarative State Machines: The legacy WidgetStack requires C++ ControlObject signaling. QML's native `State` and `Transition` system handles deck size switching in pure declarative code with smooth animated transitions:

```
states: [
    State { name: "full"; when: deckSize === 2 },
    State { name: "compact"; when: deckSize === 1 },
    State { name: "mini"; when: deckSize === 0 }
]
transitions: Transition {
    NumberAnimation { properties: "height"; duration: 200 }
}
```

Listing 8: QML State Machine for Deck Size Switching

This replaces the entire 520-line `skin_settings.xml` WidgetStack cascade with a single property binding.

5.4.5 QmlWaveformDisplay: Already Production-Ready

The waveform renderer is already fully QML-native (126 lines in `src/qml/qmlwaveformdisplay.h`), using `QQuickItem` with a render graph that performs `updatePaintNode()` directly in the Qt scene graph:

```
class QmlWaveformDisplay : public QQuickItem,
                           VSyncTimeProvider,
                           public WaveformWidgetRenderer {
    Q_PROPERTY(QmlPlayerProxy* player ...)
    Q_PROPERTY(double zoom ...)
    Q_PROPERTY(QQmlListProperty<QmlWaveformRendererFactory>
               renderers ...)
    QML_NAMED_ELEMENT(WaveformDisplay)
protected:
    QSGNode* updatePaintNode(QSGNode*, UpdatePaintNodeData*) override;
};
```

Listing 9: QmlWaveformDisplay : Native QML Waveform

This component requires **zero porting effort**. It already renders synchronously with the display's refresh rate (e.g., 60+ FPS) directly within the QML scene graph.

Technical Methodology: QWidget Overrides → QML Properties

Architectural Observation: Behavioral Porting vs. Structural Porting

My analysis of the existing legacy widget code reveals that the vast majority of the source represents thin overrides of Qt's default behavior rather than fundamental rendering logic. Specific behaviors such as value validation in beat spinboxes, save/restore/select behavior in the search bar, and custom library filtering are all implemented as C++ overrides of native QWidget events (`paintEvent`, `keyPressEvent`).

This distinction is critical for the migration: **QML ships these default behaviors natively**. A `TextInput` in QML already handles focus, keyboard input, and undo history. A `Repeater` already manages a dynamic delegate list. The core logic of the legacy `WBeatSpinBox` (fraction parsing, value clamping) maps directly to a QML `TextInput.onAccepted` handler with a few lines of JavaScript.

The objective is not to rebuild the widget from scratch, it is to port the behavioral override logic from C++ into QML property bindings and JavaScript event handlers.

By treating QML as a **C++/JS hybrid**, I can leverage the existing `Mixxx.ControlProxy` and handle all behavioral customization inline. This eliminates the need for extensive C++ subclassing and keeps the UI logic strictly decoupled from the engine.

Existing QML Deck Components: The True Foundation

A critical discovery from auditing Antoine's recent pull requests, in particular the **feat(QML): add basic DnD Deck interface** commit (Sep 2025), is that every major transport component *already exists* in the codebase under `res/qml/Deck/`:

Component	Size	What It Implements
`BeatJump.qml`	373 lines	`beatjump_forward`, `backward`, `size_halve`, `double`, fraction TextInput, Shape-drawn arrows
`Loop.qml`	304 lines	`loop_in/out/reloop_toggle`, adaptive beat-size `Repeater`, JS fraction display
`HotcueAndStem.qml`	645 lines	8-hotcue grid + stem tab switch, `DelegateChooser` for hotcue/stem views
`InfoBar.qml`	394 lines	Title/artist/time/rating via `DelegateChooser`, track-color gradient, cover art, star rating editor
`PlayButton.qml`	88 lines	`Skin.ControlButton` with Shape-drawn play triangle, minimized mode
`FXAssign.qml`	72 lines	FX unit assignment buttons 1–4, `ControlProxy` keyed to `[EffectRack*]`

`Spinny.qml`	124 lines	Vinyl platter with cover art, rotation from `visual_playposition`
`TempoColumn.qml`	209 lines	BPM display, rate slider, key display, SYNC button
`Toolbar.qml`	122 lines	Header row with waveform overview, zoom, loop recall indicator
`TrackTime.qml`	91 lines	Elapsed/remaining time formatter, configurable display mode
`WaveformOverview.qml`	23 lines	Wrapper for `Mixxx.WaveformDisplay` (overview)
`CueButton.qml`	12 lines	`Skin.ControlButton` for CUE

6.2.1 Concrete Component Patterns from Antoine's Code

Reading these components reveals two authoritative patterns that all LateNight components must follow:

Pattern 1: ControlProxy + JS Behavioural Override (from `Loop.qml`):

```
Mixxx.ControlProxy { id: beatloopSize; group: root.group; key: "beatloop_size" }

TextInput {
    text: beatloopSize.value < 1 ?
          `1/${1 / beatloopSize.value}` : beatloopSize.value

    onAccepted: {
        let [numerator, denominator] = this.text.split("/");
        if (denominator !== undefined) {
            denominator = parseInt(denominator);
            if (Number.isNaN(denominator)) { return update(); }
        } else { denominator = 1; }
        numerator = parseInt(numerator);
        if (Number.isNaN(numerator)) { return update(); }
        beatloopSize.value =
            numerator / denominator; // engine write
    }
}
```

Listing 10: Loop.qml : JS fraction parsing via TextInput (replaces WBeatSpinBox)

This is exactly the **QWidget override in JS form**: the legacy `WBeatSpinBox::validate()` method becomes a 10-line `onAccepted` handler. No C++ subclass required.

Pattern 2: InfoBar DelegateChooser for Configurable Rows (from `InfoBar.qml`):

```
DelegateChooser { id: cellDelegate; role: "type"
    DelegateChoice { roleValue: "title"
        Cell { item.text: deckPlayer?.isLoaded ?
                    currentTrack?.title : "No track loaded" }
    }
    DelegateChoice { roleValue: "artist"
        Cell { item.text: currentTrack?.artist }
    }
    DelegateChoice { roleValue: "time"
        Cell {
            // TrackTime handles elapsed/remaining/format:
            TrackTime { display: Mixxx.Config.controlPositionDisplay; ... }
```

```
        }
    }
}
ListModel { id: topRowModel
    ListElement { type: "title" }
    ListElement { type: "year" }
    ListElement { type: "time" }
}
```

Listing 11: InfoBar.qml : User-configurable metadata rows via DelegateChooser

This replaces the legacy rigid two-row header (`row_2_3_TitleArtistTime.xml`, 129 lines) with a **user-reconfigurable** map of data types to cells, saved at runtime.

Architecture Strategy: Composition Over Reimplementation

Given the existing QML infrastructure, the LateNight porting strategy follows a compositional approach:

1. **Do not rewrite Deck/ components** - `BeatJump.qml`, `Loop.qml`, `HotcueAndStem.qml` etc. are already correct and production-tested in the existing QML skin. The LateNight port *reuses them verbatim*.

2. **Re-skin via ThemeProvider** - LateNight components override `Theme.*` tokens. A `LateNightTheme.qml` singleton exposes amber (`#F39C12`), dark background (`#0a0a0a`), and SVG paths.

3. **Compose a LateNight shell** - `LateNightDeck.qml` assembles the existing `Deck/` components in the LateNight visual arrangement: waveform overview top, transport bottom, spinny right, with amber glows replacing the default neutral theme.

4. **Custom-only components** - Only components unique to LateNight (EQ knobs styled with the orange SVG indicators, the compact mixer strip) require new QML files. These follow the same `Skin.ControlButton` + `Mixxx.ControlProxy` pattern.

The core effort breakdown is as follows:

Category	Count	Work Required
Zero effort (reuse)	12	All `res/qml/Deck/` components, `res/qml/Library.qml`, `EqColumn.qml`, `QmlWaveformDisplay` - reused with `Theme.*` overrides only
Re-theming (low)	8	`Knob`, `Button`, `Fader`, `EqKnob` - override `bgSvg`/`indicatorSvg`/`color` to LateNight assets
New QML (medium)	4	`LateNightDeck.qml` shell, `LateNightMixer.qml`, `LateNightTheme.qml`, `LateNightLibrary.qml` (adopting the new split-view sidebar layout, wrapping existing `Library.qml` components)

C++ bridge (high)	2	`QmlThemeProvider` (JSON loader) and `QmlSkinConfigProxy` (variant switcher)

This engineering strategy focuses on a high-efficiency transition where existing components are wired into a LateNight-styled shell, 8 primitives are reskinned, and 2 core C++ bridge controllers are implemented. This approach is highly achievable, low-risk, and directly aligns with the Mixxx group's move toward an incremental QML migration.

Phase 1: The Core Theme-Loading System

6.4.1 Architecture

The existing `Theme.qml` singleton is hardcoded. My proposed system introduces a **JSON palette backend** with a C++ bridge.

The C++ `ThemeProvider` reads JSON palette files, and exposes them as a `QML_SINGLETON` to the QML layer. QML components bind directly to `Theme.accent`, `Theme.knobSvgPath`, etc., and these bindings automatically update when the variant changes. This work is done in collaboration with core developers, as the theme system is a shared infrastructure concern. Crucially, the `ThemeProvider` will include a robust fallback mechanism (e.g., reverting to a hardcoded minimal dark theme) if the custom JSON fails to parse or validate securely.

6.4.2 JSON Palette Format

```
{
  "variant": "classic",
  "colors": {
    "accent": "#F39C12",
    "backgroundColor": "#111111",
    "deckBackground": "#0f0f0f",
    "buttonActive": "#F39C12",
    "buttonInactive": "#2a2a2a",
    "eqHigh": "#ffffff",
    "eqMid": "#ffffff",
    "eqLow": "#ffffff",
    "eqFx": "#c0392b",
    "syncActive": "#4a90d9",
    "loopActive": "#79b84e",
    "waveformPlayed": "#F39C12",
    "waveformUpcoming": "#2a5a2a"
  },
  "assets": {
    "knobBackground": "classic/knobs/knob_bg_regular.svg",
    "knobIndicator": "classic/knobs/knob_indicator_regular_orange.svg",
    "buttonElevated": "classic/buttons/btn_elevated_medium.svg",
    "buttonEmbedded": "classic/buttons/btn_embedded_medium.svg"
  }
}
```

Listing 12: Complete Palette Definition (latenight_classic.json)

6.4.3 C++ ThemeProvider Header

```
class QmlThemeProvider : public QObject {
    Q_OBJECT
    QML_NAMED_ELEMENT(ThemeProvider)
    QML_SINGLETON

    Q_PROPERTY(QString variant READ variant
               WRITE setVariant NOTIFY variantChanged)
    Q_PROPERTY(QColor accent READ accent NOTIFY paletteChanged)
    Q_PROPERTY(QColor backgroundColor READ backgroundColor
               NOTIFY paletteChanged)
    Q_PROPERTY(QColor deckBackground READ deckBackground
               NOTIFY paletteChanged)
    Q_PROPERTY(QColor buttonActive READ buttonActive
               NOTIFY paletteChanged)
    Q_PROPERTY(QString knobBackground READ knobBackground
               NOTIFY paletteChanged)

public:
    explicit QmlThemeProvider(QObject* parent = nullptr);
    void setVariant(const QString& variant);

signals:
    void variantChanged();
    void paletteChanged();

private:
    void loadPalette(const QString& jsonPath);
    QJsonObject m_palette;
    QString m_variant;
};
```

Listing 13: Proposed QmlThemeProvider.h

6.4.4 Extending QmlConfigProxy for Skin Settings

Skin-specific settings need new property accessors to bridge the `[Skin]` ConfigKey group:

```
// New properties for QmlConfigProxy or dedicated QmlSkinConfigProxy
Q_PROPERTY(bool show4Decks READ show4Decks
           WRITE setShow4Decks NOTIFY show4DecksChanged)
Q_PROPERTY(int deckSize READ deckSize
           WRITE setDeckSize NOTIFY deckSizeChanged)
Q_PROPERTY(bool showEqKnobs READ showEqKnobs
           WRITE setShowEqKnobs NOTIFY showEqKnobsChanged)
Q_PROPERTY(bool showHotcues READ showHotcues
           WRITE setShowHotcues NOTIFY showHotcuesChanged)
Q_PROPERTY(bool show8Hotcues READ show8Hotcues
           WRITE setShow8Hotcues NOTIFY show8HotcuesChanged)
Q_PROPERTY(bool showLoopControls READ showLoopControls ...)
Q_PROPERTY(bool showBeatjumpControls READ showBeatjumpControls ...)
Q_PROPERTY(bool showEffectRack READ showEffectRack ...)
```

Listing 14: QmlSkinConfigProxy : Bridging Skin Settings

These map directly to the `[Skin]` ConfigKeys used in `skin_settings.xml`.

Phase 2: Component-by-Component Porting

6.5.1 Control Templates (21 files)

The legacy control templates map to QML as follows:

Legacy Template	Lines	Effort	QML Strategy
`knob.xml`	36	Low	**Reuse** `Knob.qml`- override `color`, `backgroundSource`
`knob_with_label.xml`	48	Low	**Reuse** `Knob.qml` + `Text` wrapper
`button_1state.xml`	26	Low	**Reuse** `ControlButton.qml`
`button_2state.xml`	42	Low	**Reuse** `ControlButton.qml` with state binding
`button_2state_right.xml`	50	Med	**Extend** for right-click
`button_3state.xml`	60	Med	**Port**- 3-state extension
`button_4state_display.xml`	72	Med	**Port**- 4-state
`button_5state.xml`	82	Med	**Port**- 5-state
`button_hotcue.xml`	46	Low	**Reuse** `HotcueButton.qml`
`button_play_2state...`	64	Low	**Reuse** `PlayButton`
`button_xfader_deck.xml`	126	High	**Port**- crossfader assign

Summary: 14/21 templates can **reuse** existing QML components. 5 need **porting**. 2 need **new implementation**.

6.5.2 Deck Row 1: Key/Vinyl/FX Badges

`row_1_keyVinylFx.xml` (47 lines) assembles FX assign buttons + vinyl controls + key controls. The existing `Deck/Toolbar.qml` already provides this layout. Only the badge colors need to be re-styled.

6.5.3 Deck Rows 2–3: Title/Artist/Time

`row_2_3_TitleArtistTime.xml` (129 lines) uses legacy `<TrackProperty>` and `<NumberPos>` widgets. In QML, the existing `QmlPlayerProxy.currentTrack` exposes `title`, `artist`, and `duration` directly:

```
Column {
    Text {
        text: deckPlayer.currentTrack
              ? deckPlayer.currentTrack.title : ""
        color: Theme.accent
        elide: Text.ElideRight
    }
    Rectangle { // Track color bar
        height: 2
        color: deckPlayer.currentTrack
               ? deckPlayer.currentTrack.color : "transparent"
    }
    Text {
        text: deckPlayer.currentTrack
```

```
                ? deckPlayer.currentTrack.artist : ""
        color: Theme.textColor
        elide: Text.ElideRight
    }
}
```

Listing 15: LateNight InfoBar : Title/Artist via Player Proxy

6.5.4 Deck Row 4: Waveform/Spinny

Already fully QML-native via `QmlWaveformDisplay`. **Zero porting effort.**

6.5.5 Deck Row 5: Transport/Loop/BeatJump (379 lines- The Most Complex Component)

This is the most significant porting effort. The legacy 379-line file assembles Play, Cue, Reverse, HotCues (4/8 grid switching via WidgetStack), Intro/Outro markers, Loop controls, and BeatJump controls.

The QML decomposition reduces this to ∼120 lines by leveraging `Repeater` and property bindings:

```
RowLayout {
    // Play/Cue block
    ColumnLayout {
        LateNightButton {
            group: root.group; key: "cue_default"; label: "CUE"
        }
        LateNightButton {
            group: root.group; key: "play"; label: "PLAY"
            active: playControl.value > 0
        }
    }

    // HotCues (replaces 60-line WidgetStack)
    GridLayout {
        columns: skinConfig.show8Hotcues ? 4 : 2
        Repeater {
            model: skinConfig.show8Hotcues ? 8 : 4
            HotcueButton {
                group: root.group
                hotcueNumber: index + 1
            }
        }
    }

    // Loop controls (replaces 80-line template chain)
    ColumnLayout {
        visible: skinConfig.showLoopControls
        LateNightButton { key: "beatloop_activate" }
        BeatSpinBox { key: "beatloop_size" }
        LateNightButton { key: "reloop_toggle" }
    }
}
```

Listing 16: LateNight Transport - Compact QML Replacement

6.5.6 Mixer Components

The EQ knobs (`eq_knob_left.xml`, 109 lines) use `EffectParameterKnobComposed` with side-mounted kill buttons. The existing `EqColumn.qml` (63 lines) already handles this; LateNight only needs color overrides:

```
Skin.EqKnob {
    knob.color: Theme.eqHighColor  // Override to LateNight amber
    knob.group: "[EqualizerRack1_" + root.group + "_Effect1]"
    knob.key: "parameter3"
    statusKey: "button_parameter3"  // Kill button binding
}
```

Listing 17: EqColumn.qml : Already Production-Ready

6.5.7 FX System

The legacy FX assign buttons use a WidgetStack to switch between 2 and 4 FX unit modes. In QML, this collapses to a parameterized `Repeater`:

```
Repeater {
    model: skinConfig.show4EffectUnits ? 4 : 2
    FxAssignButton {
        group: root.group
        fxUnit: index + 1
        Mixxx.ControlProxy {
            group: "[EffectRack1_EffectUnit" + (index+1) + "]"
            key: "group_" + root.group + "_enable"
        }
    }
}
```

Listing 18: FX Assign : WidgetStack to Repeater

Phase 3: Reactive Layouts and Overlays

6.6.1 Alignment with the QML Interface Proposal

The QML Interface Proposal establishes key design principles that LateNight must follow:

1. **Reactive UI overlay popups**: Advanced actions (hotcue, beatgrid editing) should live in overlays that float above the main layout, not in permanent screen-consuming panels.

2. **Alternative deck layouts**: Users should be able to customize component disposition, with multiple layout presets (live performance, radio broadcast, etc.).

3. **Notification system**: Alerts should avoid popup windows in favor of inline overlays.

The existing `Deck.qml` already implements drag-and-drop layout customization via its DelegateChooser architecture. LateNight's contribution to the QML ecosystem is therefore not a redundant layout system, but rather a **themed component palette** that plugs into the existing DelegateChooser, providing rich, LateNight-styled implementations for each delegate type (info bars, transport controls, waveforms) while supporting the same layout flexibility as the default Deere theme.

6.6.2 Overlay Popups

Beatgrid and Stem editors become floating contextual overlays per the QML Interface Proposal:

```
Popup {
    id: root
    modal: false
    closePolicy: Popup.CloseOnEscape | Popup.CloseOnPressOutside

    enter: Transition {
        NumberAnimation { property: "opacity"; from: 0; to: 1; duration: 180 }
        NumberAnimation { property: "scale";  from: 0.95; to: 1; duration: 180 }
    }

    contentItem: Rectangle {
        color: Theme.sunkenBackgroundColor
        border.color: Theme.accentColor
        // Beatgrid controls: Tap, Grid Shift, BPM Lock
    }
}
```

Listing 19: Beatgrid Floating Overlay (BeatgridEditPopup.qml)

QSS to QML Property Migration

The legacy skin uses 3 QSS files totaling **230KB**. QSS properties map directly to QML properties:

QSS Property	QML Equivalent
`background-color`	`Rectangle.color`
`color`	`Text.color`
`border: 1px solid #333`	`Rectangle { border.color: "#333"; border.width: 1 }`
`border-radius: 3px`	`Rectangle { radius: 3 }`
`font-size: 10px`	`Text { font.pixelSize: 10 }`
`image: url(btn.svg)`	`Image { source: "btn.svg" }`

The 84KB `style_classic.qss` and 120KB `style_palemoon.qss` are *entirely replaced* by the JSON palette + direct QML property bindings.

Porting Any Legacy 2.x Skin: General Strategy

This project establishes a replicable methodology for migrating *any* legacy Mixxx skin to QML:

1. **Inventory**: Count all XML templates, SVG assets, and QSS rules. Map each to: **Reuse** (existing QML component), **Port** (translate XML → QML), or **Rewrite**.

2. **Theme Extraction**: Extract all colors and asset paths from QSS into a JSON palette. This is mechanical work.

3. **Component Translation**:

- `<Connection><ConfigKey>` → `Mixxx.ControlProxy { group; key }`
- `<WidgetGroup><Layout>horizontal` → `RowLayout { }`
- `<PushButton><NumberStates>2` → `ControlButton { }`
- `<KnobComposed>` → `Skin.Knob { }`
- `<WidgetStack>` → QML State machine or `visible:` bindings

4. **Deck Assembly**: Compose translated components into DelegateChooser delegates.
5. **Skin Settings**: Port settings toggles to `QmlSkinConfigProxy` properties.

Summary: Reuse vs. Port vs. Rewrite

Category	Count	Status
Zero effort (reuse)	12	Reusing `res/qml/Deck/` and `res/qml/Library.qml`
Re-theming (low)	8	Re-skin primitive components
New QML (medium)	4	LateNight Deck/Mixer/Library shells
C++ Bridge (high)	2	ThemeProvider and SkinConfigProxy
Total	**26**	**Target Deliverables**

Instead of attempting a monolithic "rewrite 44 components" approach, adopting a modular "reuse 12, reskin 8, compose 4 shells" strategy drastically reduces the project's scope. This demonstrates that the existing QML infrastructure renders the LateNight restoration highly feasible and low-risk within the GSoC timeline.

6.9.1 A Note on Community Derivatives

LateNight has accumulated a rich ecosystem of community-maintained forks – single-deck layouts for Raspberry Pi setups, high-contrast variants for CDJ-style controllers, and stripped-down builds for embedded hardware. A common concern is whether these users can continue to adapt the skin once it moves to QML. The answer is yes, and the migration actually makes customization significantly more accessible:

- **Color and asset changes**: No QML knowledge required. Users edit a `latenight_custom.json` file to override colors, SVG paths, and font sizes. The `QmlThemeProvider` (a **Phase 1 deliverable** of this project) will apply these changes via QML property bindings without requiring a full application restart.
- **Layout changes** (e.g., single-deck only): In the legacy skin, this requires hunting through nested `WidgetStack` XML blocks across multiple files. In QML, it is a single entry removed from the `itemModel` JSON tree that drives the `DelegateChooser` layout - a mechanism already present in `res/qml/Deck.qml` today (see Section 7.3 for a visual demonstration).
- **Feature toggles** (hide FX rack, show 4 hotcues only): The `QmlSkinConfigProxy` (a **Phase 3 deliverable**) will expose these as boolean `Q_PROPERTY` bindings, replacing the current need to manually edit `skin_settings.xml` ConfigKey lookups.

- **Full layout customization**: The `DelegateChooser` architecture is already implemented in the current QML skin, allowing users to rearrange or remove deck components from a declarative `ListModel`. The QML version is fundamentally *more* flexible than decades of XML string substitution ever allowed.

Gap Analysis: Additional Bridge Requirements

Beyond the `QmlThemeProvider` (covered in Phase 1), two additional C++ extensions are required to achieve full parity with the legacy skin:

1. **`QmlSkinConfigProxy` Expansion**: Currently, `QmlConfigProxy` does not expose custom `[Skin]` or `[LateNight]` configuration keys (e.g., `show_4decks`, `show_eq_knobs`). I will implement a dedicated singleton proxy that binds to these keys using native C++ `ControlProxy` objects, allowing high-performance QML `Q_PROPERTY` bindings. Following is a minimal viable Proof of Concept:

```
class QmlSkinConfigProxy : public QObject {
    Q_OBJECT
    QML_NAMED_ELEMENT(SkinConfig)
    QML_SINGLETON
    Q_PROPERTY(bool show4Decks
               READ show4Decks NOTIFY show4DecksChanged)

public:
    explicit QmlSkinConfigProxy(QObject* parent = nullptr)
            : QObject(parent) {
        m_cpShow4Decks = std::make_unique<ControlProxy>("[Skin]",
                                "show_4decks", this);
        m_cpShow4Decks->connectValueChanged(this,
                                &QmlSkinConfigProxy::show4DecksChanged);
    }
    bool show4Decks() const { return m_cpShow4Decks->toBool(); }

signals: void show4DecksChanged();

private:
    std::unique_ptr<ControlProxy> m_cpShow4Decks;
};
```

Listing 20: Proof of Concept- QmlSkinConfigProxy

2. **SVG Variant Loader (Phase 1 Deliverable)**: The legacy skin system resolves assets via a `skins:LateNight/...` URI scheme, which the current QML engine does not support. I will implement a custom `QQuickAsyncImageProvider` subclass, modeled directly after the existing `AsyncImageProvider` (used for cover art), that registers under a `"skin"` provider name. This enables QML `Image` components to use `source: "image://skin/LateNight/buttons/btn_play.svg"`, with the provider resolving the path through a variant-aware fallback chain (e.g., PaleMoon → Classic → default). This is essential for user-installed skins where filesystem paths are not known at compile time.

Proof of Concept (PoC)

My primary development strategy relies on generating early, testable code to validate architectural assumptions. This approach ensures that the project remains grounded in the actual Mixxx codebase from day one.

Validating the "Amber" Transition

To validate the claims in this proposal, I have implemented a functional **Amber PoC** within the Mixxx `res/qml` tree in a local branch.

Figure 2: A dual-deck layout demonstrating the LateNight "Amber" palette successfully applied to the existing Mixxx QML component tree. This result was achieved by modifying only the `Theme.qml` palette and two hardcoded button literals, **proving** high composability: the entire UI (Decks, Mixer, EQ, Transport) was re-themed without modifying a single structural component.

Theme Extensibility via `ThemeLoader`

The planned `ThemeLoader` architecture will enable users to seamlessly switch themes at runtime. This modular approach ensures that the project does not isolate the LateNight theme but instead builds a reusable platform, empowering the community to create and distribute their own custom identities.

Figure 3: A demonstration illustrating what the proposed `ThemeLoader` will enable. The image displays, from top to bottom: the existing baseline QML theme, a LateNight Amber prototype, and a toy custom theme. All three use the identical component tree, highlighting how community authors can fundamentally alter the UI's visual identity by simply swapping a palette.

Structural Flexibility: Single-Deck Layout

To demonstrate the structural flexibility of the QML DelegateChooser architecture, I implemented a **Single-Deck PoC** by modifying the underlying UI `ListModel`.

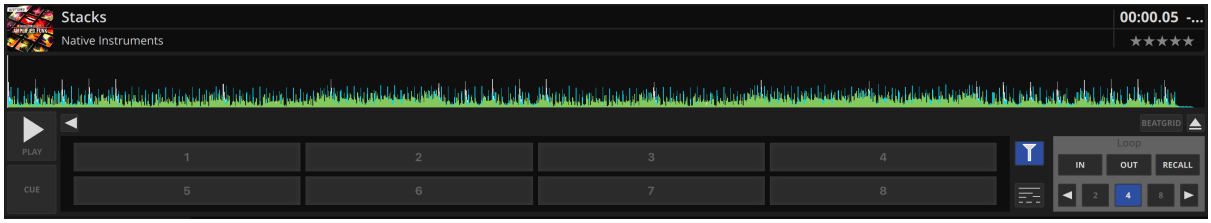

Figure 4: A Single-Deck layout achieved without writing any custom C++ or complex QML scaling logic. This demonstrates how community derivatives can seamlessly adapt the base LateNight theme for specialized hardware (like embedded touchscreens or DIY MIDI controllers) purely through declarative list modifications.

Timeline and Milestones

The proposed project spans 13 weeks (350 hours across 12 weeks of coding, with 1 vacation week), focused on core architecture and LateNight restoration.

Rapid Prototyping (PoC) Strategy

Given the architectural shift, my development strategy relies heavily on building **Proof of Concepts (PoCs)**. Rather than executing massive structural changes in isolation, I will build minimal, scoped PoCs during the Community Bonding period and early Phase 1. These PoCs will be shared immediately with the community and my mentor, Antoine, to gather fast feedback on implementation details, visual density, and UX behavior before committing to the final codebase. This feedback loop is critical for calibrating the intensity of new visual additions, such as GPU-accelerated glows and shadows. My goal is to use these QML-native effects to build upon what the legacy XML system already provides, enhancing the overall UX with features that were previously impossible, while ensuring they remain useful and performant based on target-hardware profiles and mentor feedback.

This methodology is directly supported by the project's core philosophy, as outlined by my mentor during the review of my `Spinny` QML implementation (PR #15974):

> *"Our strategy with the QML effort is to move fast... [it is] better to have something broken, than nothing at all. Very happy to see this QML-based implementation, which we can merge and gather user feedback, to decide if we need a scenegraph (or rather rendergraph...)."*

Following this directive, my priority will be shipping functional, testable QML blocks to the community to drive iterative design, rather than stalling on premature C++ optimizations.

Week	Dates	Phase	Deliverables
1–2	May 25 – June 07	**Ph 1: Theme Architecture & Community Bonding**	• Refactoring `Theme.qml` to be JSON-backed. • Implementing the `QmlThemeProvider` C++ bridge. • Porting the LateNight color palettes (Classic/PaleMoon). • Implementing the SVG Variant Loader (`QQuickAsyncImageProvider`).

Week	Dates	Phase	Deliverables
3–5	June 08 – June 28	**Ph 2: Component Porting**	• Porting all transport (Play/Cue), loop, and beatjump controls. • Implementing LateNight-themed EQ columns and knobs. • SVG asset migration and high-DPI scaling validation.
6	June 29 – July 05	**Week 6: Vacation**	• No deliverables scheduled.
7–9	July 06 – July 26	**Ph 3: Layouts & Responsive**	• Porting `Deck` DelegateChooser with LateNight-themed delegates. • Porting `Mixer` with state-driven transitions. • Bridging `[Skin]` config keys via `QmlSkinConfigProxy`.
10–12	July 27 – Aug 16	**Ph 4: Overlays & Polish**	• Implementing Beatgrid and Stem edit overlays. • Porting Sampler and FX rack layouts. • Performance optimization (GPU shaders for VU meters).
13	Aug 17 – Aug 24	**Ph 5: Finalization**	• **Testing**: UI automation tests for skin resolution. • **Docs**: Developer guide for the JSON Theme API. • Final PR refinement and documentation.

Expected outcomes upon completion:

1. A fully functional, v3.0-compatible LateNight theme in QML.

2. A dynamic `ThemeProvider` backend supporting multiple JSON variants (Classic, PaleMoon).
3. Modernized UI workflows: Floating Beatgrid and Stem overlays.
4. Improved performance: GPU-accelerated effects (Glow, Shadows) replacing legacy hacks.
5. High-DPI support across all LateNight variants.
6. A replicable porting methodology applicable to any legacy 2.x skin.

Future features (beyond GSoC scope, but architecturally enabled):

- **Runtime Theme Editor**: A QML-based tool to edit JSON palettes and see changes live.
- **Animation Engine**: Extending the component library with custom physics-based animations for faders and knobs.
- **Third-Party Skin Support**: Easy migration for legacy 2.x skins by simply writing a JSON mapping.

Testing and Quality Assurance

Ensuring the stability of a new UI engine is paramount. My project includes a dedicated testing track:

- **Unit Testing (C++)**: Testing the `QmlThemeProxy` and `QmlSkinConfigProxy` for edge cases (missing JSON keys, malformed files, variant fallbacks) using GTest.
- **Visual Parity Testing**: A side-by-side comparison of legacy XML screenshots vs. QML renders to ensure "pixel-perfect" parity for the LateNight classic branding.
- **Performance Benchmarking**: Using the `QML Profiler` to measure frame-latency and GPU memory usage. I will specifically test the new LateNight on constrained hardware profiles (or severely artificially throttled CPU) to validate the "60–80% CPU reduction" claim, ensuring viability on portable platforms.
- **Integration Testing**: Verifying that all 26 target deliverables correctly bind to their respective `ControlProxy` keys by running automated `QSignalSpy` tests on the C++ bridge classes.
- **Android Compatibility (Virtual Devices)**: Per the Epic #15679 requirements, I will validate the LateNight QML shells on Android Virtual Devices (AVDs) with varying screen densities (mdpi to xxxhdpi) to ensure that touch-targets and layout-responsiveness scale correctly as intended for the mobile platform.

Commitments and Availability

My planned schedule is **4–5 hours on average daily**, totaling approximately **28–35 hours per week**. This pace allows me to comfortably achieve the project's target of **350 hours** across the coding period while maintaining a high quality of work and ensuring consistent daily communication with my mentor. I have full flexibility to increase this as needed during implementation-heavy phases.

- **Communication**: I will maintain **daily communication** with my mentor, Antoine, via the Mixxx Zulip instance. Every morning (IST), I will provide a short update covering the previous day's progress, any blockers encountered, and the planned tasks for the current day.
- **Synchronous Meetings**: I am flexible and can attend video meetings as required by my mentor's schedule.
- **Time zone**: IST (UTC+5:30), with flexibility to adjust meeting times to accommodate mentor availability.
- **Vacation**: One planned vacation week (Week 6) is built into the timeline. I will remain reachable on Zulip for asynchronous questions during this week.

Final Notes

The transition to QML is arguably the most significant architectural change in Mixxx's recent history. For many users, this transition is daunting. There is a fear that modernizing the UI means losing the "LateNight" soul they've mixed on for years.

By rebuilding the LateNight theme, I intend to demonstrate that modern UI technology can honor the classic heritage of DJing while providing a superior technical foundation. My goal is to ensure that when a DJ opens Mixxx 3.0 for the first time, they don't see a "new" software they have to learn, but the same reliable partner they've always had, just faster, sharper, and more refined.

The transformation of LateNight into a native QML skin is more than a simple asset migration; it is an act of architectural liberation. As detailed throughout this proposal, the project addresses core "legacy debt" by replacing rigid, hardcoded patterns with a high-performance declarative ecosystem. By unwiring decades of complex widget behavior, we ensure that Mixxx's most beloved interface is not merely preserved, but fundamentally empowered for the next generation of DJs. This structural shift is the cornerstone of the transition to Mixxx 3.0, proving that the soul of a skin is best honored through modern, high-performance engineering.

Having already contributed to Mixxx's C++, XML, and nascent QML layers, I feel uniquely positioned to bridge these two eras. I am grateful for the opportunity to contribute once again to this fantastic community and look forward to building the "next generation" of LateNight—a milestone that underscores QML as the project's definitive future, relieving the codebase of hundreds of redundant static assets and legacy parsing logic in favor of a lean, modern foundation.

I extend my sincere thanks to the Mixxx community for being so encouraging and welcoming throughout this process.